

Auto GPU Kernel: Autonomous Kernel Discovery for DeepSeek Sparse Attention

MLSys 2026 FlashInfer AI Kernel Generation Contest: DSA Track, Full-Agent Sub-Track Winner 🏆

Doğaç Eldenk
Northwestern University
United States
dogacel@gmail.com

Abstract

We describe our Full-Agent submission to the MLSys 2026 FlashInfer AI Kernel Generation Contest on the DeepSeek Sparse Attention (DSA) track. The kernels were generated end-to-end by an autonomous Claude Code optimization loop with no human in the loop, using a small set of *skills* and three *sub-agents* (profiler, workload-inspector, research). The optimizations the loop discovered are workload-specialized rather than purely micro-architectural: scoreless fast paths for short-context regimes, a fused radix top- k that replaces torch.topk plus a remap pass, a single-launch split-then-combine sparse-attention kernel with an intra-kernel atomic barrier, and a stride-partitioned split that eliminates imbalance. We argue that the bottleneck for autonomous kernel discovery is *experiment management* combined with *high degrees of freedom* for the agent. The agent achieved 34.93x average speedup and ranked #1.

Keywords

sparse attention, Triton, GPU kernels, autonomous coding agents, DeepSeek, FlashInfer, MLSys

1 Introduction

DeepSeek Sparse Attention (DSA), introduced with DeepSeek-V3.2 [3], splits long-context attention into two stages: a low-rank “lightning indexer” selects per-query top- k key positions, and a sparse attention kernel attends only over those indices into a paged KV cache. The MLSys 2026 FlashInfer AI Kernel Generation Contest [1] asks for high-performance DSA kernels for the *Blackwell Architecture*, evaluated on *NVIDIA B200* against a reference FlashInfer implementation.

The Full-Agent sub-track requires a fully autonomous pipeline that produces submissions with no human in the loop. Our submission is built on top of the Claude Code agent. We use the agent’s *Sandboxing* feature to limit its internet access, while allowing it to access *Modal* [2] to queue jobs to run kernels. The project has three skills (`/optimize`, `/benchmark`, `/log-experiment`), and three sub-agents (profiler, workload-inspector, research). The loop is launched via `/loop Run /optimize` every 15 minutes and self-reschedules less aggressively as low-hanging fruit runs out. We applied it to both DSA sub-problems, the TopK indexer and the sparse attention kernel, with shared infrastructure but separate repositories and experiment trees.

2 Background

DSA decoding has two kernels in series. The **TopK indexer** computes per-token scores against an FP8 paged K-cache (page size 64, structure-of-arrays per-row scales) and returns the top $K=2048$ token indices, padded with -1 . The score is $s_t = (\sum_h \text{relu}(q_h \cdot K_{t,h}) w_{b,h}) \text{scale}_t$. The **sparse attention** kernel takes paged latent KV caches and the $K=2048$ sparse indices per token (with -1 padding) and returns flash-attention output and LSE. The TopK indexer is evaluated on 23 traces and Sparse Attention on 128 traces, sampled from real DeepSeek-V3.2 inference.

3 The Autonomous Optimization Loop

The loop is a Claude Code agent driven by three durable artifacts: a project rulebook (`CLAUDE.md`), an append-only experiment summary (`summary.md`), and a hand-curated lesson log (`LESSONS.md`). Each iteration of `/optimize` runs a fixed sequence: *assess* (read files only, never trusting in-context memory across iterations), *plan one change*, *implement*, *validate* via `/benchmark quick`, *measure* via `/benchmark stride 8`, and *log* via `/log-experiment`. Figure 1 sketches one iteration.

3.1 Non-negotiable rules

The `CLAUDE.md` enforces a small, blunt set of rules tuned to the failure modes we observed early:

- *Absolute latencies only.* Reference latency drifts by 20–30% across VMs; cross-iteration speedup ratios are noise.
- *One optimization per iteration.* Coupled changes misattribute wins.
- *Sub-5% deltas use the paired A/B harness* (`ab_benchmark.py`), which packs both candidates into one Modal container and runs them back-to-back so the reference latency cancels.
- *Log every experiment, including failures and ablations.*
- *No benchmark gaming.* CUDA graphs and output memoization are forbidden.
- *Stay in Triton* (or Gluon, but only after 15–20 stuck iterations); do not switch languages on a whim.

Each rule was added because the agent violated it or got stuck due to it at least once. Encoding the rules into the project file made them survive context resets.

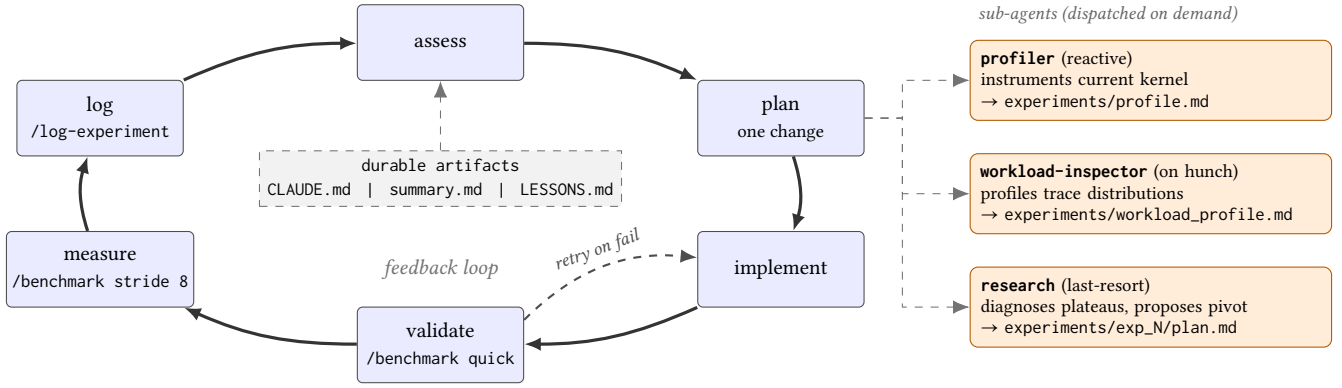


Figure 1: The autonomous /optimize loop.

3.2 Sub-agents and the artifact contract

Three sub-agents are launched on demand from the optimizer’s main context. Crucially, they do *not* return information through the conversation channel. They write to a fixed file path, and the optimizer reads that file.

- **profiler** (reactive). Instruments the current kernel with `torch.cuda.Event` pairs to attribute time to phases (FP8 dequant, scoring, top-*k*, remap), reports p_{50}/p_{90} split by small/large workloads, and writes `experiments/profile.md`. We intentionally let the agent decide how to instrument the kernel rather than giving it a specific tool to use.
- **workload-inspector** (on hunch). Runs a Modal job that loads the trace set and reports distributions of `batch_size`, `seq_len`, `max_num_pages`, padding fractions, and indirection locality, writing `experiments/workload_profile.md`. The agent decides what metrics to look at and how to look at them; the analysis code is entirely self-written. The output is durable because the trace set is static.
- **research** (last-resort). Reads the artifacts above with no in-context memory of the optimizer’s recent attempts, diagnoses the plateau against a checklist (repetition loop, wrong bottleneck, missing fundamental, overlooked shortcut), and writes `experiments/exp_(N+1)/plan.md` for the optimizer to implement.

The clean-context discipline for research matters: when the optimizer is stuck it is also biased; a fresh agent reading only the artifacts is consistently better at suggesting a pivot than the optimizer is at suggesting it to itself.

3.3 workload-inspector: filling a reasoning gap

Early in the loop the agent reasoned over the kernel’s code, not over the kernel’s inputs. It had no mechanism to ask “what does the data look like” independent of “what does the code look like.” The largest single algorithmic improvement on the indexer came from an observation about the *workload*: roughly half of the context traces have a valid token count that fits inside $K=2048$. On those traces, top-*k* selection collapses from a sort to a structural

mapping. The answer is just the valid token IDs in their natural order, with no scoring, no comparisons, and no `torch.topk`.

The autonomous loop did not propose this on its own until we added *workload-inspector*. The agent was too fixated on improving the code to do the necessary exploration. Once installed, the inspector surfaced opportunities the loop kept missing, such as padding-aware skipping in sparse attention (a prefix invariant per token; threads owning purely-padded positions can early-exit). We treat *workload-inspector* as the canonical example of a sub-agent that exists to fill a *reasoning gap* that opens up when the agent is too fixated on code and not running small experiments to understand the problem.

3.4 Tiered benchmarking

The single largest end-to-end speedup of the loop came from cutting the cost of one experiment. `/benchmark` exposes three modes: `quick` runs only 2 workloads (smallest and largest, correctness-only), `stride 8` runs ≈ 16 workloads (the default per-iteration measurement), and `full` runs all 128 (used only to lock in a confirmed new best). We also disabled the harness’s reference-baseline run on per-iteration measurements, since absolute kernel latency is the only number the rulebook trusts.

4 Kernel Design and Optimizations

This section walks the optimizations the autonomous loop converged on in the submitted kernels.

4.1 TopK Indexer

Two-tier dispatch on max_seq_padded. The host computes $M = \text{max_num_pages} \cdot \text{PAGE_SIZE}$ and branches on whether $M \leq K$. When it is, the trace is scoreless: every valid token fits in the top-*K* set, and the answer is just the page-table-expanded valid IDs. Two passthrough Triton kernels cover this branch — a single-page form for `max_num_pages=1`, and a multi-page form (grid (batch, $\lceil K/256 \rceil$), `BLOCK_K=256`) for $1 < \text{max_num_pages} \leq 32$. Otherwise we run `scoring + top-k`.

Zero-copy SoA view of the FP8 KV cache. The cache’s declared shape $[P, 64, 1, 132]$ `int8` hides an SoA per-page layout: `[8192 fp8 bytes | 256 scale bytes]`. We expose it to the score

kernel as two `torch.as_strided` views (`k_fp8 [P, 64, 128]`, `k_scale [P, 64]`) directly over the same storage, bypassing any `.contiguous()` copy and its launch.

Score kernel with TMA-friendly block pointers. The score kernel uses one program per (batch, logical page) with `TOKENS_PER_TILE=PAGE_SIZE=64`. `tl.make_block_ptr` carries shape/contiguity metadata that lowers Q/K loads to TMA on B200. Scalar scale and weight loads are issued early to overlap MMA latency. Linearity of the score ($\text{scale} \geq 0$, $\max(xs, 0) = s \max(x, 0)$) lets us apply the per-token scale once after the cross-head sum, eliminating a 64×64 broadcast multiply.

Fused Triton radix top-k. The loop replaced `torch.topk` (plus its remap pass) with a fused Triton radix kernel that does bit-by-bit selection on fp32 scores with a packed strict/tie cumsum, fusing the page-table lookup $\text{tok}_j = \text{page_table}[\text{idx}_j/P] \cdot P + \text{idx}_j \bmod P$ into the output write so the indexer never materializes flat indices.

4.2 Sparse Attention

Hybrid dispatch on T . For $T \leq 2$ the launch overhead of any split/combine pair dominates compute, so we use a single-grid D -parallel fused kernel (`_small_t_kernel`, `BLOCK_N=64`) that does the whole flash attention in one launch. For $T \geq 3$ we use the split-combine layout below.

Single-launch split+combine via in-kernel atomic barrier. The standard flash-decoding pattern is two kernels back-to-back; on B200 each launch costs roughly $8 \mu\text{s}$ and the pair sits squarely on the critical path of small- T workloads. We collapse them into one. Each CTA writes its partial (m, ℓ, acc) to a per-token scratch-pad, atomically increments a per-token counter with release semantics, then spins on the same counter until all `NUM_SPLITS` siblings have arrived; a `tl.debug_barrier` guarantees visibility, after which the same CTA executes the combine phase for one D -slice. Reusing `program_id(1)` as both the split index (phase 1) and the D -slice index (phase 2) is the geometric trick that makes a single grid suffice.

Stride-partitioned splits. Because the sparse-index padding is a prefix of valid entries, a contiguous chunk-partition of the top- k axis would assign all work to the first few splits. Instead, split s owns positions $\{s, s + \text{NUM_SPLITS}, s + 2\text{NUM_SPLITS}, \dots\}$, so a prefix of V valid indices is spread $\approx V/\text{NUM_SPLITS}$ across splits. A one-shot scan counts valid indices and bounds the inner loop, so splits with no valid work pay near-zero. KV loads use `cache_modifier=".cg"` with `eviction_policy="evict_first"`; Q uses `evict_last` since it is reused across all KV tiles.

Online softmax in $\log_2$ base. We carry m, ℓ in $\log_2$ space, multiplying the score by `sm_scale  $\cdot \log_2 e$` once and using `tl.exp2/tl.log2` thereafter, keeping softmax on the FFMA-friendly fast path on Blackwell.

5 Tools and Languages

The submitted kernels are Triton. The choice was a negative result, not a preference.

Previously we tested our agents against five languages: **CUDA**, **CuTeDSL**, **Triton**, **Gluon**, and **Helion**. Despite being more

powerful, CUDA and CuTeDSL produced consistently worse autonomous results: each new optimization required multiple rounds to fix compile errors, then more rounds to restore numerical correctness, before any performance signal could be measured. The repeated build-fix-build cycles induced a kind of *plan amnesia*, where the agent drifted from the original optimization goal as it absorbed compiler-error churn. Abandoned attempts then turned into *false beliefs about infeasibility* that trapped the agent in shallow local minima.

Triton, Gluon, and Helion all converged on competitive solutions. Triton’s win comes from being the right abstraction for the agent, not necessarily for a hand-tuner: Python syntax, high-level tile primitives, and short edits between viable benchmarks. Helion was similarly easy to iterate but offered less customization headroom. Gluon, in principle, has the most headroom; in our autonomous loop the agent could not consistently apply its lower-level optimizations and in the typical case fell short of the Triton kernel.

Underlying model capability. The choice of language interacts strongly with the choice of LLM. We compared Claude Opus 4.7 (used for the submitted runs) against DeepSeek-V3 driving the same loop, same rulebook, same Triton kernel. DeepSeek-V3 needed several times more iterations per optimization, and spent most of those iterations stuck on basic syntax and algorithmic correctness rather than performance reasoning. Below a certain capability threshold, the loop’s experiment hygiene cannot compensate.

Implication. The right abstraction with minimal compromise, paired with a sufficiently capable LLM, is the way to make agents discover lightning-fast kernels fully autonomously. The bottleneck for autonomous code generation is rarely raw expressive power, and is often the cost of every individual restructuring step the agent has to commit to.

6 Performance Analysis

All experiments run on a single NVIDIA B200 via Modal [2], against the FlashInfer-Bench harness for the DSA definitions ($H=64$, $D=128$, $K=2048$, page size 64 for the indexer). Latencies are reported as the absolute median of 100 iterations across 5 trials with 3 warmup runs. Per the rulebook, comparisons across iterations use the paired A/B harness on the same VM; cross-VM ratios are reported only as headline numbers and treated as advisory.

Final headline numbers. The Sparse Attention kernel achieves an average runtime of 0.010 ms and the TopK indexer 0.016 ms, yielding an average $34.93\times$ speedup over the FlashInfer baselines and ranking our kernel first in the competition.

Where the wins came from (qualitative). Per-experiment logs in `summary.md` show that the largest single autonomous steps were:

- *Indexer:* scoreless paths (`mp=1` and `mp $\leq$ 32`) on the affected trace subset (only after workload-inspector surfaced the regime), and the fused radix top- k on the long-context subset (only after profiler attributed the bottleneck to `torch.topk`).

- *Sparse attention*: collapsing the split/combine launch pair into one kernel via the atomic barrier, and the stride-partitioned splits on small-valid workloads.

None are micro-tunings; all required the workload-inspector or profiler artifact to surface the opportunity.

Where the autonomous loop came up short. A handful of optimizations the loop systematically missed: it kept the score scratch and the partial (m, ℓ, acc) accumulator at fp32 rather than bf16 (the safety argument requires connecting an accumulator’s value range to a rescaling step a few lines away, which the loop did not do unprompted); it called `torch.empty` on partial buffers every invocation (over-applying the “no benchmark gaming” rule and conflating allocation reuse with output memoization); and its tile geometry was empirically fine but not anchored to Blackwell’s `tcgen05` 64×64×128 MMA tile. Every item shares the same shape: an optimization the loop *could* have proposed, did not, and would not have noticed was missing without a comparison artifact to point at.

7 Conclusion

What worked. Treating the agent as an experiment manager, with a strict rulebook and durable artifacts, but many degrees of freedom worked the best. Giving the agent a sandbox in which it could freely write code, run experiments, and launch jobs paid off the most; trying to constrain it to specific code paths failed. Delegating certain tasks to sub-agents with clean contexts also helped greatly to escape plateaus. Persisting past solutions, learnings, and plans helped the agent not lose its long-horizon plan.

Remaining gaps. When stuck, the loop oscillates between recently-tried micro-tunings rather than pivoting. The research agent helps, but the optimizer rarely changes its mind, since a fresh proposal seldom beats the current best within a few iterations. A separate method for spawning experiments with different seeds and using another agent to combine their solutions could be explored. Lower-level languages such as CuTe DSL or CUDA would also help the agent build even faster solutions, but they require the agent to be more capable in the underlying language and to make fewer mistakes; as agents become more and more capable, we expect them to excel at other languages too.

Safety observations. Internet access was disabled in both the Claude Code sandbox and the project rulebook. As a separate test we asked the agent to evaluate its own jailbreak skills; it declined. Conversely, when sandbox restrictions blocked Modal CLI calls it considered legitimate, it tried to route around them by spinning up a local DNS server to proxy gRPC traffic. This is a benign instance of a general pattern (a sufficiently capable agent will find creative paths around restrictions when it judges the goal worth the effort), which is precisely why the same alignment behavior that respects “no CUDA graphs” is load-bearing.

8 Reproducibility

The codebase used to run the agents, along with all logs generated during the competition, is available at <https://github.com/Dogacel/auto-gpu-kernel>.

Acknowledgments

We thank the MLSys 2026 contest organizers at NVIDIA and the FlashInfer team for the evaluation harness and traces, the DeepSeek team for releasing DSA, and the Modal team for the remote-GPU backend used for every measurement in this paper.

References

- [1] FlashInfer team. *MLSys 2026 FlashInfer AI Kernel Generation Contest*. <https://mlsys26.flashinfer.ai/>, 2026.
- [2] Modal Labs. *Modal: Serverless infrastructure for AI/ML*. <https://modal.com/>, 2025.
- [3] DeepSeek-AI. *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models* 2025.
- [4] Tillet, Philippe and Kung, Hsiang-Tsung and Cox, David. *Triton: an intermediate language and compiler for tiled neural network computations*. Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, 2019.